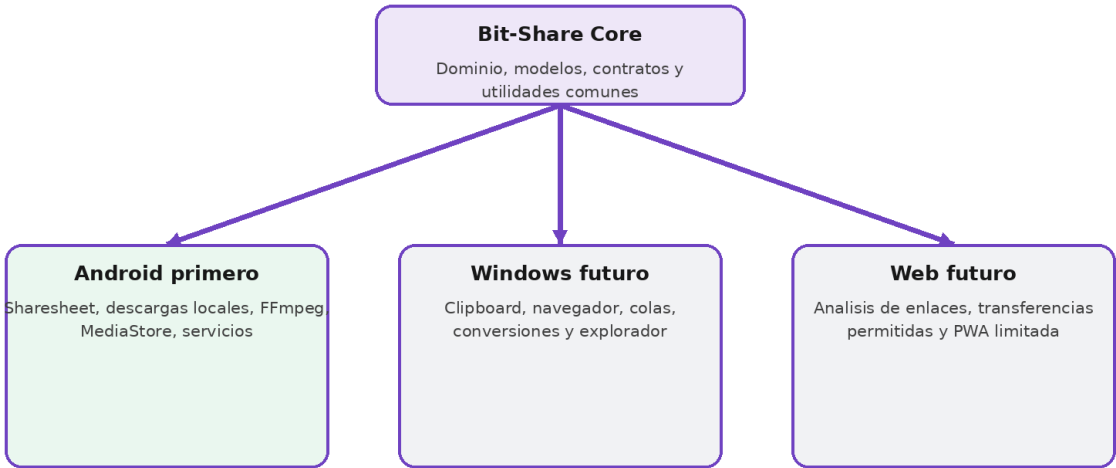


BIT-SHARE

Documentacion tecnica, planificacion y prompt maestro

Aplicacion Flutter multiplataforma - Desarrollo inicial Android

Un solo repositorio Flutter, experiencias distintas



Principio: compartir dominio y contratos, no forzar la misma GUI ni las mismas capacidades en todas las plataformas.

Proposito del documento

Entregar a otra IA o equipo de desarrollo una especificacion ejecutable para construir Bit-Share de forma modular, segura y realista. El documento diferencia claramente entre la experiencia deseada y las capacidades que Android y las plataformas externas permiten.

Preparado para BitStation

Version 1.0 - 29/07/2026

Control del documento

Campo	Valor
Nombre del producto	Bit-Share
Marca paraguas	BitStation
Tipo	Aplicacion Flutter multiplataforma con integracion nativa por plataforma
Prioridad actual	Android
Plataformas futuras	Windows y Web
Version del documento	1.0
Fecha	29 de julio de 2026
Estado	Especificacion inicial para arquitectura y MVP
Idioma	Espanol

Convenciones

- REQUISITO: comportamiento obligatorio para el producto.
- RECOMENDACION: decision preferida, revisable si aparece una alternativa mejor.
- RESTRICCION: limite impuesto por Android, una API externa, seguridad o politica de distribucion.
- FUERA DE ALCANCE: no se implementa en el MVP Android.

Lectura recomendada

- [Resumen ejecutivo](#)
- [Veredicto sobre cookies y sesiones](#)
- [Arquitectura](#)
- [Plan de desarrollo](#)
- [Prompt maestro](#)
- [Criterios de aceptacion](#)

1. Resumen ejecutivo

Bit-Share sera una aplicacion Flutter con nucleo compartido y experiencias diferentes por plataforma. La primera entrega sera Android y funcionara como destino del menu Compartir: el usuario comparte un enlace, texto o archivo desde otra aplicacion; Bit-Share analiza el contenido; descarga o procesa la multimedia cuando sea tecnicamente posible y autorizado; finalmente pregunta si debe guardarla en la galeria, conservarla como archivo o volver a compartirla con otra aplicacion.

El producto puede integrar yt-dlp y FFmpeg en Android mediante una capa nativa Kotlin comunicada con Flutter. Las descargas prolongadas deben ejecutarse como trabajo visible y recuperable, con notificacion, cancelacion, reintentos y persistencia de estado. Los resultados deben guardarse mediante MediaStore o el selector de documentos de Android.

Decision critica

Android no permite que Bit-Share lea temporalmente las cookies, tokens o cache privada de Instagram, YouTube, TikTok, Facebook, X, Twitch u otra aplicacion instalada. Cada aplicacion funciona dentro de su propio sandbox. Esta limitacion sigue existiendo aunque todo ocurra localmente y aunque el usuario sea propietario del dispositivo.

Para contenido autenticado, la arquitectura debe admitir conectores por proveedor. Cada conector puede usar OAuth oficial, APIs autorizadas, enlaces o archivos compartidos y, solo cuando la plataforma lo permita, una sesion creada dentro del propio espacio de Bit-Share. Iniciar sesion oficialmente no equivale a recibir las cookies completas ni garantiza una descarga universal de contenido privado.

El MVP debe concentrarse en la experiencia principal: recibir contenido, resolver el proveedor, descargar contenido compatible, convertirlo, guardarlo y compartirlo. La autenticacion avanzada se incorporara por proveedor y nunca se presentara como acceso ilimitado.

1.1 Matriz de viabilidad

Capacidad	Viabilidad	Decision
Aparecer en el menu Compartir de Android	SI	Implementar ACTION_SEND y ACTION_SEND_MULTIPLE.
Recibir URL, texto, imagen, audio o video	SI	Procesar Intent y URI con permisos temporales.
Descargar mediante yt-dlp	SI, segun proveedor	Motor reemplazable y actualizado.
Convertir y unir pistas con FFmpeg	SI	Priorizar remux antes que recodificacion.
Guardar en galeria o Descargas	SI	Usar MediaStore y almacenamiento temporal privado.
Volver a compartir el archivo descargado	SI	Usar FileProvider/URI y Android Sharesheet.
Leer cookies privadas de otras apps	NO en Android estandar	No implementar extraccion cross-app.
Autenticar al usuario por plataforma	PARCIAL	OAuth/SDK/API por proveedor.
Descargar cualquier contenido privado sin limites	NO garantizable	Depende de permisos, API, DRM y terminos.
Actualizar yt-dlp sin actualizar toda la app	SI, con controles	Manifiesto firmado, hash, rollback y canal estable.

2. Vision del producto

2.1 Propuesta de valor

Bit-Share reduce a una sola accion el flujo de conservar o redistribuir multimedia que el usuario puede compartir desde otra aplicacion. El producto no intenta convertirse en una red social ni en un navegador completo; actua como una herramienta local, rapida y orientada a tareas.

- Recibir contenido desde el Sharesheet sin copiar y pegar manualmente.
- Detectar automaticamente el proveedor y el tipo de contenido.
- Elegir calidad, formato y destino cuando corresponda.
- Procesar localmente con yt-dlp y FFmpeg.
- Guardar en galeria, guardar como archivo o compartir inmediatamente.
- Mantener una cola visible y recuperable para trabajos largos.

2.2 Identidad y experiencia

Nombre visible: Bit-Share. Nombre de paquete recomendado: com.bitstation.bitshare. El guion se conserva en la marca y la etiqueta de Android, pero no en el identificador Java/Kotlin.

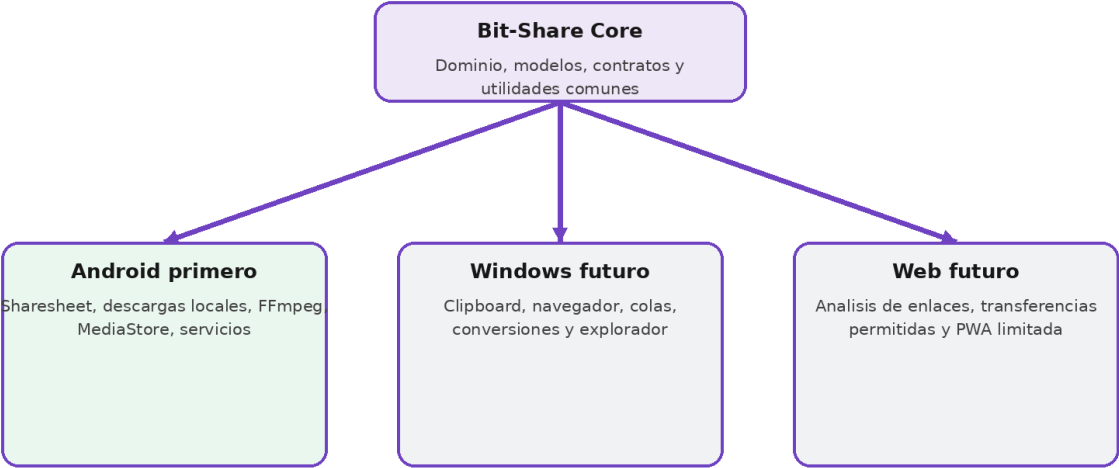
La version Android debe sentirse como una extension: interfaz compacta, accion inmediata, pocas pantallas y retorno rapido a la aplicacion de origen. La aplicacion completa puede abrirse para consultar historial, cola, ajustes, proveedores y almacenamiento.

2.3 Principios de producto

Principio	Aplicacion practica
Local primero	Descarga, conversion, metadatos y archivos temporales se procesan en el dispositivo.
Minima friccion	Compartir -> Bit-Share -> elegir destino, sin navegar por menus complejos.
Seguridad por diseno	No capturar contraseñas ni extraer sesiones privadas de otras apps.
Modularidad	Cada proveedor y cada plataforma se implementan como adaptadores reemplazables.
Degradacion controlada	Cuando una descarga no sea posible, explicar la causa y ofrecer acciones validas.
Mantenibilidad	Versiones fijadas, actualizaciones verificadas, rollback y pruebas automatizadas.

2.4 Alcance por plataforma

Un solo repositorio Flutter, experiencias distintas



Principio: compartir dominio y contratos, no forzar la misma GUI ni las mismas capacidades en todas las plataformas.

Regla de arquitectura

Se comparte el dominio, los modelos y los contratos. No se obliga a Android, Windows y Web a tener la misma GUI ni las mismas capacidades. Cada plataforma puede implementar herramientas diferentes sobre el mismo nucleo conceptual.

3. Alcance del MVP Android

3.1 Incluido

- Registro de Bit-Share como destino para ACTION_SEND y ACTION_SEND_MULTIPLE.
- Recepcion de texto, URL, imagen, audio y video mediante Intent.
- Previsualizacion del contenido compartido.
- Resolucion del proveedor y normalizacion de URL.
- Descarga de contenido compatible mediante yt-dlp o descarga HTTP directa.
- Seleccion de calidad y formato cuando existan alternativas.
- Union, remux, extraccion de audio y conversion mediante FFmpeg.
- Guardado en MediaStore.Video, MediaStore.Audio, MediaStore.Images o MediaStore.Downloads.
- Opcion de compartir el archivo resultante usando el Sharesheet.
- Cola de descargas con progreso, velocidad, ETA, pausa logica, cancelacion y reintento.
- Notificacion persistente para tareas visibles y prolongadas.
- Actualizador seguro para yt-dlp y componentes interpretados compatibles.
- Historial local y limpieza de archivos temporales.
- Arquitectura preparada para conectores autenticados por proveedor.

3.2 Fuera de alcance inicial

- Version funcional para Windows.
- Version funcional para Web/PWA.
- Sincronizacion en nube entre dispositivos.
- Servidor central que reciba cookies, contraseñas o archivos del usuario.
- Extraccion de cookies, bases de datos o cache privada de aplicaciones instaladas.
- Evasion de DRM, cifrado, paywalls o controles de acceso.
- Descarga universal de cualquier contenido privado.
- Automatizacion oculta mediante Accessibility Service.
- Root, Shizuku, ADB o explotacion de vulnerabilidades como requisito normal.

3.3 Supuestos

- Android minimo recomendado: API 26 para simplificar soporte; objetivo inicial: Android 10 a Android 16, validando cambios de Android 17 antes de publicar.
- Arquitectura principal de dispositivos: arm64-v8a. x86_64 solo para emuladores de desarrollo.
- El usuario solo procesara contenido propio, autorizado o cuya descarga sea legal y permitida.
- La distribucion puede dividirse en edicion publica compatible con Google Play y edicion avanzada para distribucion privada, sin mezclar politicas ni capacidades ocultas.

4. Veredicto sobre cookies, cache y autenticacion

4.1 Requisito deseado

La experiencia deseada consiste en que el usuario comparta un video desde una cuenta a la que ya tiene acceso y Bit-Share pueda reutilizar temporalmente esa autenticacion para descargar la multimedia, sin enviar credenciales a ningun servidor y sin pedir un inicio de sesion adicional.

4.2 Restriccion de Android

Resultado tecnico

Ese acceso directo no es implementable en un telefono Android estandar. El sandbox asigna a cada aplicacion un espacio privado y un identificador de usuario diferente. Bit-Share no puede abrir los archivos internos, la base de datos de cookies, el WebView, los tokens o la cache de otra aplicacion aunque el usuario haya pulsado Compartir.

El Intent de compartir solo contiene los datos que la aplicacion origen decide exponer: texto, URL, ClipData, URI de contenido, tipo MIME y, en algunos casos, permisos temporales de lectura sobre ese URI. No incluye la sesion privada de la aplicacion origen.

4.3 Opciones de autenticacion permitidas

Modo	Que recibe Bit-Share	Ventajas	Limites
Contenido compartido	URL, texto o URI concedido por la app origen	Experiencia mas rapida y segura	No aporta cookies privadas.
OAuth / SDK oficial	Token limitado por scopes	Credenciales no pasan por Bit-Share	No equivale a cookie ni permite descargar todo.
API oficial del proveedor	Datos y operaciones expuestas	Compatible con politicas	Cobertura limitada y revisiones.
Sesion web propia de Bit-Share	Cookies creadas dentro del almacenamiento de Bit-Share	Permite una sesion separada cuando el sitio lo admite	No reutiliza la app oficial; algunos proveedores bloquean WebView.
Archivo de cookies importado por el usuario	Cookie jar elegido explicitamente	Puede ayudar en una edicion avanzada	Riesgo alto, caduca, puede infringir terminos; no extraer automaticamente.

4.4 Regla de implementacion

Frontera obligatoria de seguridad

```

NO implementar:
- Lectura de /data/data/<otra_app>/...
- Extraccion de cookies mediante root, ADB o Accessibility
- Captura de contraseñas o tokens mostrados en pantalla
- Copia oculta de bases de datos de Chrome, Instagram, TikTok, etc.

SI implementar:
- Datos que la app origen comparte por Intent
- OAuth/SDK oficial por proveedor
- Sesión propia de Bit-Share solo cuando sea legal y compatible
- Boveda local cifrada para tokens emitidos a Bit-Share
- Mensajes claros cuando un recurso privado no sea accesible

```

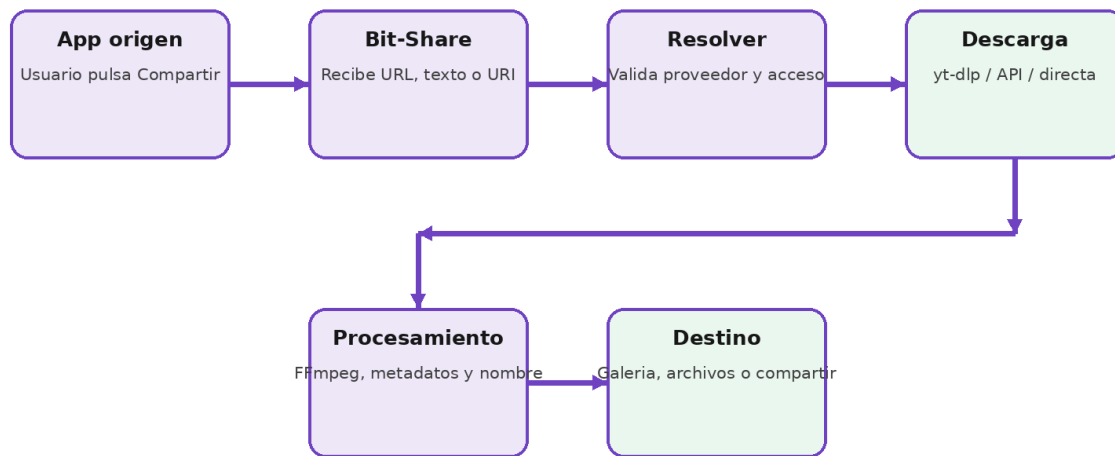
4.5 Acceso privado realista

La descarga de contenido autenticado debe resolverse proveedor por proveedor. Un conector declarara sus capacidades: publico, propio, autorizado por OAuth, archivo directo, contenido compartido como URI, o no soportado. La interfaz nunca debe decir "sin restricciones"; debe mostrar el nivel de acceso verificado para el recurso concreto.

Estado de acceso	Significado para el usuario
PUBLIC	El recurso se puede resolver sin sesion.
SHARED_FILE	La aplicacion origen entrego directamente un archivo o URI legible.
AUTHORIZED_API	La API oficial confirma que Bit-Share puede acceder.
BITSHARE_SESSION	Una sesion propia de Bit-Share permite resolverlo.
AUTH_REQUIRED	Hace falta autorizar un conector compatible.
NOT_EXPOSED	La plataforma no expone el recurso a terceros.
DRM_OR_PROTECTED	Existe proteccion que Bit-Share no debe evadir.

5. Flujos de usuario

Flujo principal Android - Bit-Share



Nota: el Sharesheet entrega datos compartidos; no entrega cookies privadas de la app origen.

5.1 Flujo A - URL publica

1. El usuario pulsa Compartir en la aplicación origen y elige Bit-Share.
2. ShareReceiverActivity recibe ACTION_SEND con text/plain.
3. El sistema normaliza la URL, elimina parámetros de seguimiento configurables y detecta el proveedor.
4. El conector consulta metadatos y formatos disponibles.
5. Bit-Share muestra miniatura, título, duración, calidad y tamaño estimado.
6. El usuario elige Guardar, Compartir después de descargar o Cancelar.
7. La descarga se ejecuta en segundo plano visible y el archivo final se publica en MediaStore.

5.2 Flujo B - La app origen comparte un archivo

Si el Intent contiene content:// con permiso temporal de lectura, Bit-Share debe copiar o transmitir el contenido desde ContentResolver a su almacenamiento temporal. No debe resolver rutas reales inseguras ni asumir que el URI pertenece al sistema de archivos.

- Si el formato final ya es compatible, permitir guardado o reenvío inmediato.
- Si hace falta conversión, crear un trabajo FFmpeg con estimación de espacio.
- Conservar el permiso solo durante el ciclo requerido; no persistirlo salvo que Android lo permita y sea necesario.

5.3 Flujo C - Recurso autenticado

8. El conector detecta AUTH_REQUIRED.
9. Bit-Share ofrece Conectar cuenta solo si existe un método oficial o una sesión propia soportada.
10. El usuario completa el flujo en Custom Tab, app oficial o interfaz autorizada.
11. El conector almacena un token emitido para Bit-Share, cifrado con Android Keystore.
12. Se reintenta la resolución del recurso y se informa el resultado real.
13. Si la plataforma no expone el archivo, se cancela sin intentar extraer cookies de otra app.

5.4 Flujo D - Guardar o compartir

Accion	Resultado
Guardar en galeria	Publicar video, audio o imagen en la coleccion MediaStore adecuada.
Guardar como archivo	Abrir ACTION_CREATE_DOCUMENT o escribir en Downloads.
Compartir ahora	Crear URI segura y abrir Android Sharesheet.
Solo descargar	Conservar en la ubicacion configurada y registrar historial.
Cancelar	Detener procesos, borrar temporales incompletos y conservar diagnostico no sensible.

6. Requisitos funcionales

ID	Requisito	Prioridad	Modulo
FR-001	Registrar receptor de compartir	MUST	Android
FR-002	Aceptar ACTION_SEND y ACTION_SEND_MULTIPLE	MUST	Android
FR-003	Validar esquema, host y MIME	MUST	Core/Android
FR-004	Mostrar previsualizacion antes de descargar	MUST	Flutter
FR-005	Resolver formatos mediante adaptador	MUST	Core
FR-006	Descargar con progreso y cancelacion	MUST	Android
FR-007	Unir audio y video	MUST	FFmpeg
FR-008	Extraer audio opcionalmente	SHOULD	FFmpeg
FR-009	Guardar con MediaStore	MUST	Android
FR-010	Compartir archivo resultante	MUST	Android
FR-011	Persistir cola e historial	MUST	Core/Android
FR-012	Recuperar trabajos tras cierre	SHOULD	Android
FR-013	Actualizar yt-dlp de forma verificada	SHOULD	Android
FR-014	Rollback automatico	MUST si hay updater	Android
FR-015	Conectores autenticados por proveedor	LATER	Adapters
FR-016	Boveda local de tokens	MUST si hay OAuth	Android
FR-017	Limpiar temporales	MUST	Android
FR-018	Diagnostico exportable sin secretos	SHOULD	Core
FR-019	Canal stable/nightly opcional	LATER	Updater
FR-020	Modo solo Wi-Fi	SHOULD	Android

6.1 Reglas de negocio

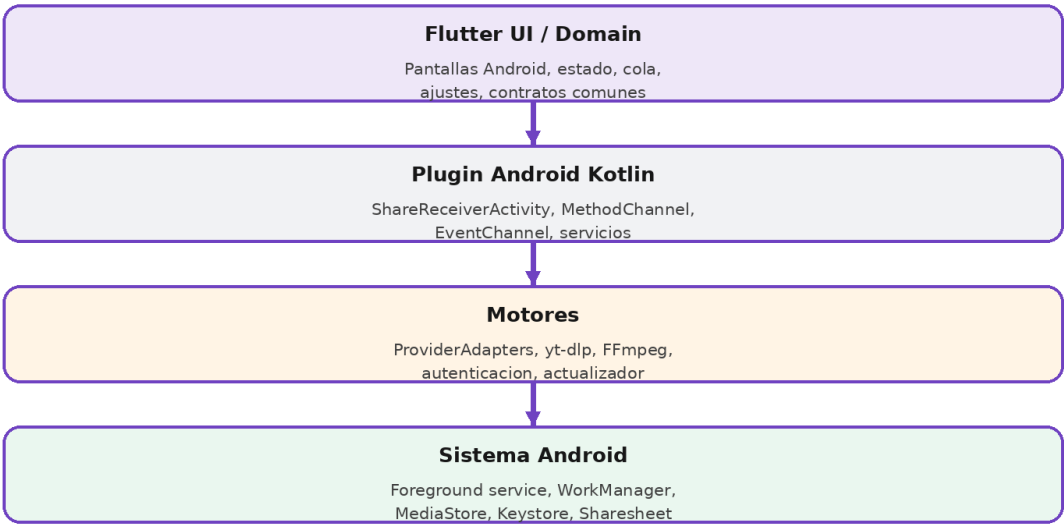
- RB-001: no iniciar una descarga hasta conocer el espacio libre estimado o una cota conservadora.
- RB-002: priorizar remux con -c copy; recodificar solo si el usuario lo solicita o es necesario por compatibilidad.
- RB-003: no conservar cookies, tokens ni cabeceras en logs.
- RB-004: cada proveedor declara capacidades y restricciones; no existe un comportamiento universal.
- RB-005: si el origen comparte un archivo, ese archivo tiene prioridad sobre volver a descargarlo.
- RB-006: los trabajos deben ser idempotentes o disponer de identificadores para evitar duplicados accidentales.
- RB-007: una actualizacion del motor solo se activa despues de verificar firma, hash y prueba basica.
- RB-008: conservar al menos una version anterior funcional del motor.
- RB-009: no presentar una opcion de autenticacion si el conector no puede utilizarla para el recurso.
- RB-010: las acciones de guardar y compartir deben ejecutarse unicamente tras confirmacion del usuario.

6.2 Requisitos no funcionales

Categoria	Objetivo
Rendimiento	Abrir la hoja de accion en menos de 800 ms en dispositivo medio, sin inicializar FFmpeg hasta necesitarlo.
Fiabilidad	Ninguna descarga parcial debe publicarse como archivo final.
Seguridad	Secretos cifrados, logs saneados, URLs validadas y binarios/versiones verificados.
Privacidad	Procesamiento local por defecto; ninguna telemetria de URL completa sin consentimiento.
Mantenibilidad	Separacion estricta entre UI, dominio, adaptadores y plataforma.
Compatibilidad	Pruebas en Android 10, 12, 14, 15 y 16; revisar Android 17.
Accesibilidad	Etiquetas semanticas, contraste AA, navegacion por lector de pantalla y botones grandes.
Internacionalizacion	Espanol base; cadenas externalizadas desde el primer sprint.

7. Arquitectura tecnica

Arquitectura modular propuesta



7.1 Capas

Capa	Responsabilidad	Tecnologia
Presentacion	Pantallas, hoja de accion, cola, historial, ajustes	Flutter/Dart
Aplicacion	Casos de uso, orquestacion, estado	Dart
Dominio	Modelos y contratos sin dependencias de Android	Dart puro
Infraestructura comun	Persistencia, serializacion, logging saneado	Dart/Kotlin
Plugin Android	Intents, servicios, MediaStore, Keystore, notificaciones	Kotlin
Motores	yt-dlp, FFmpeg, HTTP, metadatos	Python/Native/Kotlin segun decision
Adaptadores de proveedor	Deteccion, autenticacion y resolucion	Modulos independientes

7.2 Comunicacion Flutter - Android

Usar MethodChannel para comandos discretos y EventChannel para eventos de progreso. Las operaciones largas no deben depender de que el proceso Flutter permanezca en primer plano. Kotlin debe persistir el trabajo y rehidratar el estado cuando Flutter vuelva a conectarse.

Contrato inicial de canales

```
MethodChannel: bitshare/methods
- initializeEngine()
- inspectSharePayload(payloadId)
- startDownload(request)
- cancelDownload(taskId)
- retryDownload(taskId)
- publishToMediaStore(taskId, destination)
- shareResult(taskId)
- checkComponentUpdates()

EventChannel: bitshare/events
```

- shareReceived
- metadataResolved
- downloadProgress
- processingProgress
- taskCompleted
- taskFailed
- componentUpdateState

7.3 Componentes Android

Componente	Funcion
ShareReceiverActivity	Punto de entrada translucido/compacto desde Android Sharesheet.
MainActivity	Aplicacion completa: historial, cola y ajustes.
DownloadCoordinator	Orquesta resolucion, descarga, procesamiento y publicacion.
BitShareWorker	Trabajo persistente con WorkManager.
DownloadForegroundService	Notificacion y control para operacion visible prolongada, si aplica.
MediaStoreRepository	Inserta, confirma y elimina resultados en almacenamiento compartido.
SecureTokenVault	Cifra tokens con claves no exportables de Android Keystore.
ComponentUpdater	Descarga manifiesto, valida firma/hash, prueba y activa version.
ProviderRegistry	Selecciona el adaptador apropiado para cada URL/MIME.

7.4 Modelos de dominio

Esquema conceptual

```
enum InputKind { url, text, imageUri, audioUri, videoUri, multiUri }
enum AccessLevel { public, sharedFile, authorizedApi, bitShareSession,
    authRequired, notExposed, protected }
enum TaskState { queued, resolving, downloading, processing,
    publishing, completed, failed, cancelled }

SharePayload
- id
- sourcePackage?
- action
- mimeType?
- text?
- uris[]
- receivedAt

DownloadTask
- id
- payloadId
- providerId
- selectedFormat
- destination
- state
- bytesDownloaded
- totalBytes?
- errorCode?
```

8. Mapa de archivos recomendado

Estructura orientativa del monorepo

```

bit_share/
|-- apps/
|   |-- bit_share_flutter/
|       |-- lib/
|           |-- app/
|           |-- core/
|               |-- errors/
|               |-- logging/
|               |-- security/
|               |-- utils/
|           |-- domain/
|               |-- entities/
|               |-- repositories/
|               |-- usecases/
|           |-- features/
|               |-- share_receiver/
|               |-- media_inspector/
|               |-- download_queue/
|               |-- history/
|               |-- settings/
|           |-- platform/
|               |-- bitshare_channel.dart
|               |-- bitshare_events.dart
|           |-- main.dart
|       |-- android/app/src/main/kotlin/com/bitstation/bitshare/
|           |-- share/ShareReceiverActivity.kt
|           |-- bridge/BitSharePlugin.kt
|           |-- download/DownloadCoordinator.kt
|           |-- download/BitShareWorker.kt
|           |-- media/MediaStoreRepository.kt
|           |-- security/SecureTokenVault.kt
|           |-- update/ComponentUpdater.kt
|           |-- providers/ProviderRegistry.kt
|           |-- notifications/NotificationController.kt
|-- packages/
|   |-- bitshare_domain/
|   |-- bitshare_provider_api/
|   |-- bitshare_storage/
|   |-- bitshare_design_system/
|-- native/
|   |-- yt_dlp_runtime/
|   |-- ffmpeg_runtime/
|-- docs/
|   |-- ARCHITECTURE.md
|   |-- SECURITY.md
|   |-- PROVIDER_CONTRACT.md
|   |-- UPDATE_PROTOCOL.md
|   |-- TEST_PLAN.md
|   |-- CODE_MAP.md
|-- tools/
|   |-- build_android_runtime/
|   |-- sign_component_manifest/
|   |-- verify_release/
|-- README.md

```

8.1 Reglas de mantenimiento del repositorio

- No borrar ni sobrescribir código existente sin una revisión explícita; crear archivos nuevos o parches pequeños y trazables.
- Mantener CODE_MAP.md actualizado después de cada cambio estructural.
- Fijar versiones de Flutter, Dart, Gradle, Kotlin, NDK, yt-dlp y FFmpeg.
- Separar secretos de configuración y no incluirlos en Git.
- Cada adaptador de proveedor debe incluir pruebas, capacidades declaradas y documentación de sus límites.
- Toda migración de persistencia debe ser versionada y reversible cuando sea posible.

9. Integracion con Android Sharesheet

9.1 Intent filters

La actividad receptora debe declarar filtros separados y estrictos. Usar text/plain para enlaces, image/*, video/*, audio/* y, si se requiere, */* como ultimo recurso controlado. ACTION_SEND_MULTIPLE debe validarse por cantidad y tipo.

Ejemplo conceptual de AndroidManifest.xml

```
<activity
    android:name=".share.ShareReceiverActivity"
    android:exported="true"
    android:theme="@style/Theme.BitShare.Transparent">

    <intent-filter>
        <action android:name="android.intent.action.SEND" />
        <category android:name="android.intent.category.DEFAULT" />
        <data android:mimeType="text/plain" />
    </intent-filter>

    <intent-filter>
        <action android:name="android.intent.action.SEND" />
        <category android:name="android.intent.category.DEFAULT" />
        <data android:mimeType="video/*" />
    </intent-filter>
</activity>
```

Seguridad del receptor

Toda Activity exportada debe tratar el Intent como entrada no confiable. Validar URI, MIME, tamaño, número de elementos, esquema, host, permisos concedidos y tiempo de procesamiento antes de copiar datos.

9.2 Hoja de accion compacta

Bloque	Contenido
Cabecera	Miniatura, proveedor, título o nombre de archivo.
Estado	Público, archivo compartido, autenticación requerida o no soportado.
Formato	Calidad, contenedor, audio/video y tamaño estimado.
Destino	Guardar en galería, guardar como archivo, compartir después.
Acciones	Descargar, cancelar, abrir en Bit-Share.

9.3 Direct Share futuro

Bit-Share puede publicar accesos directos como "Guardar en galería" o destinos frecuentes, pero esto se implementará después de estabilizar el flujo básico. No debe confundirse con acceso directo a la sesión de la aplicación origen.

10. Motor de descarga y resolucion

10.1 Cadena de resolucion

Pipeline

```
SharePayload
-> InputNormalizer
-> ProviderRegistry
-> ProviderAdapter.inspect()
-> AccessDecision
-> FormatSelection
-> DownloadPlan
-> DownloaderEngine
-> FfmpegPlan
-> OutputPublisher
```

10.2 Estrategias de descarga

Estrategia	Uso	Prioridad
SharedUriDownloader	La app origen comparte un archivo o stream legible	1
DirectHttpDownloader	URL directa y validada a un recurso permitido	2
OfficialApiDownloader	La API devuelve un recurso descargable autorizado	3
YtDlpDownloader	Proveedor compatible y uso autorizado	4
Unsupported	No existe metodo permitido o fiable	Final controlado

10.3 Integracion de yt-dlp

La decision exacta del runtime debe validarse con un prototipo. La interfaz del dominio no debe depender de Python ni de un wrapper concreto. Dos rutas principales son: runtime Python embebido con llamada a la API de yt-dlp, o wrapper Android mantenido/fork propio. El proyecto debe evitar una dependencia sin mantenimiento o con una version de Python incompatible.

- No ejecutar comandos contruidos concatenando texto no validado.
- Usar estructuras de opciones y listas de argumentos.
- Limitar rutas de salida al sandbox temporal de Bit-Share.
- Filtrar nombres de archivo y longitud maxima.
- Desactivar listas de reproduccion por defecto; habilitarlas en una fase posterior con confirmacion.
- No imprimir cookies, cabeceras Authorization ni URLs firmadas en logs.

10.4 Formato y seleccion

El usuario debe ver opciones comprensibles, no identificadores internos de yt-dlp. Ejemplo: "1080p - MP4 - video + audio - 86 MB". Si el proveedor entrega audio y video separados, Bit-Share debe indicar que los unira sin perdida cuando sea posible.

11. Procesamiento con FFmpeg

11.1 Operaciones del MVP

Operacion	Objetivo	Politica
Remux	Cambiar contenedor sin recodificar	Primera opcion por velocidad y bateria.
Merge	Unir pista de video y audio	Usar -map y -c copy cuando codecs sean compatibles.
Extract audio	Crear M4A, MP3, Opus o WAV	Advertir si requiere recodificacion.
Transcode	H.264/AAC para compatibilidad	Solo por solicitud o necesidad.
Metadata	Titulo, artista, miniatura	Sanear texto y respetar formatos.

11.2 Gestion termica y energetica

- Priorizar copias de streams sobre recodificacion.
- Mostrar advertencia para 4K, HDR o archivos grandes.
- Permitir limitar resolucion y tasa de bits.
- No ejecutar multiples transcodificaciones pesadas en paralelo por defecto.
- Pausar o retrasar trabajos opcionalmente con bateria baja o temperatura elevada.

11.3 Archivos temporales

Cada tarea debe usar un directorio aislado: cacheDir/tasks/<taskId>. El resultado se publica con un nombre temporal y solo se marca como final cuando FFmpeg termina con codigo 0, el archivo existe, su tamano es razonable y una inspeccion basica confirma el contenedor.

12. Actualizacion interna de componentes

12.1 Objetivo

Evitar que yt-dlp quede obsoleto sin obligar a publicar una nueva version completa de Bit-Share para cada cambio de extractor. La app siempre debe incluir una version base funcional y nunca depender de una descarga inicial obligatoria.

12.2 Componentes y politica

Componente	Actualizar internamente	Decision
yt-dlp interpretado	Si, sujeto a politica y compatibilidad	Frecuente, canal stable.
Reglas/configuraciones propias	Si	Manifiesto firmado.
yt-dlp EJS u otros scripts compatibles	Si, como unidad versionada	Mantener compatibilidad declarada.
Python runtime	No normalmente	Actualizar con la aplicacion.
FFmpeg / bibliotecas .so	No para Play	Actualizar con AAB/APK.
Kotlin/Dart/DEX/JAR	No	Actualizar la aplicacion.

12.3 Protocolo seguro

- 14. Consultar manifiesto HTTPS con cache y frecuencia maxima configurada.
- 15. Verificar firma Ed25519 del manifiesto con clave publica embebida.
- 16. Comprobar version minima de app, Python, ABI y esquema.
- 17. Descargar a staging con limite de tamano y timeout.
- 18. Verificar SHA-256 antes de descomprimir.
- 19. Marcar archivos dinamicos como solo lectura cuando Android lo requiera.
- 20. Ejecutar prueba local: version, importacion y extractor de prueba controlado.
- 21. Activar mediante cambio atomico de puntero current.
- 22. Conservar previous para rollback automatico.
- 23. Registrar solo versiones y codigos; nunca datos de usuario.

Manifiesto conceptual

```
{
  "schema": 1,
  "channel": "stable",
  "generatedAt": "2026-07-29T20:00:00Z",
  "components": {
    "yt_dlp": {
      "version": "YYYY.MM.DD",
      "url": "https://updates.example/yt-dlp.zip",
      "sha256": "...",
      "minApp": "0.1.0",
      "minPython": "3.10"
    }
  },
  "signature": "base64-ed25519-signature"
}
```

Distribucion Google Play

La carga dinamica de codigo remoto es una zona de alto riesgo tecnico y de politica. La edicion para Google Play debe evitar descargar DEX, JAR, SO, FFmpeg o runtimes. Cualquier actualizacion interpretada debe someterse a revision de politica y no utilizarse para cambiar el proposito de la app.

13. Almacenamiento, seguridad y privacidad

13.1 Almacenamiento

Dato	Ubicacion
Archivos temporales	Almacenamiento interno privado de Bit-Share.
Resultados visibles	MediaStore o ubicacion elegida con Storage Access Framework.
Historial	Base local con metadatos minimos; URL completa opcional y protegida.
Tokens OAuth	Cifrados con clave de Android Keystore.
Configuracion	DataStore/SharedPreferences cifradas segun sensibilidad.
Logs	Directorio privado, rotacion y saneamiento.

13.2 Modelo de amenazas

Amenaza	Control
Intent malicioso	Validar entrada, limites, MIME, esquema, host y URI.
Path traversal	Nunca usar nombres del proveedor como ruta sin sanear.
Archivo gigante	Cuotas, comprobacion de espacio y limites configurables.
Componente actualizado alterado	Firma, hash, staging, solo lectura y rollback.
Fuga de tokens	Keystore, cero logs, borrado de memoria cuando sea viable.
SSRF / URL interna	Bloquear localhost, redes privadas y esquemas no permitidos para descargas remotas.
Archivo multimedia malformado	Procesar en sandbox de app, versiones parcheadas y timeouts.
Deep link abusivo	Lista permitida de rutas y acciones; no ejecutar comandos desde parametros.

13.3 Privacidad por defecto

- No usar un servidor de BitStation para procesar URLs o credenciales en el MVP.
- No enviar telemetria de enlaces, titulos o nombres de cuenta sin consentimiento granular.
- Ofrecer borrar historial, tokens, cookies propias y temporales.
- Mostrar claramente que una cuenta conectada es independiente de la app oficial.
- Permitir modo sesion temporal que elimina el token al completar el trabajo.

13.4 Contenido autorizado

Uso permitido

Bit-Share debe limitarse a contenido propio, autorizado, libre o cuya descarga este permitida. No debe evadir DRM, controles tecnicos ni restricciones de acceso. La disponibilidad tecnica no elimina las obligaciones legales o contractuales.

14. Diseno de interfaz Android

14.1 Pantallas

Pantalla	Objetivo	Prioridad
Share Action Sheet	Decidir formato y destino desde el Sharesheet	MVP
Resolviendo	Mostrar analisis breve y permitir cancelar	MVP
Progreso	Barra, velocidad, ETA, etapa y acciones	MVP
Resultado	Guardar, compartir, abrir ubicacion	MVP
Cola	Ver trabajos activos y pendientes	MVP
Historial	Repetir accion, compartir o borrar registro	MVP
Proveedores	Conectar/desconectar cuentas compatibles	Fase 2
Actualizaciones	Versiones de motores y canal	Fase 2
Diagnostico	Exportar reporte saneado	Fase 2

14.2 Estilo visual

Identidad recomendada: tema oscuro, acentos purpura, alto contraste y lenguaje visual tecnico de BitStation. La hoja que aparece al compartir debe ser mas simple que la aplicacion completa y no debe bloquear al usuario con configuraciones avanzadas.

- Accion principal unica y visible.
- Estados expresados con texto e icono, no solo color.
- Tamano, duracion y costo de conversion antes de comenzar cuando sea posible.
- Errores accionables: Reintentar, Autorizar, Cambiar formato, Guardar enlace o Abrir origen.

14.3 Ejemplos de mensajes

Situacion	Mensaje
URL publica	Video disponible. Elige calidad y destino.
Archivo ya compartido	La aplicacion envio el video directamente. No es necesario volver a descargarlo.
Autenticacion compatible	Conecta tu cuenta para comprobar si este recurso esta disponible para Bit-Share.
Contenido no expuesto	Puedes verlo en la aplicacion original, pero la plataforma no lo entrega a aplicaciones externas.
Proteccion	Este contenido utiliza una proteccion que Bit-Share no puede eludir.
Sin espacio	Se necesitan aproximadamente 1.8 GB adicionales.

15. Estrategia de pruebas

15.1 Piramide

Nivel	Cobertura
Unitarias Dart	Normalizacion, seleccion de proveedor, reglas de formato, estados y errores.
Unitarias Kotlin	Parsing de Intent, MediaStore, updater, boveda y notificaciones.
Contract tests	Cada ProviderAdapter contra fixtures y respuestas simuladas.
Integracion	Flutter <-> MethodChannel/EventChannel y persistencia de trabajos.
Instrumentacion Android	Sharesheet real, URLs, rotacion, background, permisos y reanudacion.
E2E	Compartir -> descargar -> convertir -> guardar/compartir.
Seguridad	Fuzz de URLs, archivos malformados, intents no confiables y updater manipulado.

15.2 Matriz minima de dispositivos

Perfil	Version	Objetivo
Emulador x86_64	Android 10	Compatibilidad base y almacenamiento.
Emulador	Android 12	Permisos y background.
Emulador	Android 14	Foreground service types y DCL.
Dispositivo arm64	Android 15	Rendimiento, bateria y MediaStore.
Dispositivo arm64	Android 16	Compatibilidad actual y page size si aplica.
Preview	Android 17	Cambios futuros de carga dinamica y servicios.

15.3 Casos forenses

- Cerrar Flutter durante una descarga y reabrir.
- Reiniciar el dispositivo con trabajo pendiente.
- Perder red y recuperarla.
- Quedarse sin espacio durante merge.
- Compartir URI que caduca al cerrar la actividad origen.
- Recibir un Intent sin MIME o con MIME falso.
- Cancelar FFmpeg y confirmar limpieza de procesos/temporales.
- Instalar una actualizacion corrupta y validar rollback.
- Token revocado o expirado durante resolucion.
- Dos solicitudes iguales recibidas en segundos.

16. Planificacion del desarrollo

16.1 Fases y entregables

Fase	Objetivo	Entregables	Salida
0 - Descubrimiento	Validar runtimes y restricciones	PoC yt-dlp, PoC FFmpeg, ADRs, riesgos	Decision Go/No-Go
1 - Esqueleto	Crear monorepo y contratos	Flutter app, plugin Kotlin, CI, CODE_MAP	Build reproducible
2 - Sharesheet	Recibir y visualizar contenido	ShareReceiver, parser, hoja de accion	Demo desde apps reales
3 - Descarga publica	Resolver y descargar URLs	ProviderRegistry, yt-dlp/direct HTTP, progreso	MVP tecnico
4 - Procesamiento	FFmpeg y resultados	Merge, remux, audio, MediaStore, compartir	MVP usuario
5 - Robustez	Background, cola e historial	WorkManager/FGS, reintentos, limpieza	Beta interna
6 - Updater	Actualizar motor con rollback	Manifiesto, firma, staging, pruebas	Beta estable
7 - Auth piloto	Un proveedor oficial	OAuth/SDK, Keystore, capacidad declarada	Validacion autentificada
8 - Publicacion	Politica, QA y privacidad	Ficha, politicas, SBOM, release	Candidato 1.0

16.2 Sprints sugeridos

Sprint	Contenido	Checkpoint
S1	Repositorio, CI, Flutter shell, plugin Kotlin, modelo SharePayload	La app compila y el canal responde.
S2	ACTION_SEND, ACTION_SEND_MULTIPLE, previsualizacion y validacion	Bit-Share aparece en Compartir y recibe datos.
S3	ProviderRegistry, metadatos y PoC yt-dlp	Resuelve al menos dos fuentes publicas autorizadas.
S4	Descarga, progreso, cancelacion y temporales	Descarga estable con cierre/reapertura.
S5	FFmpeg, MediaStore y compartir resultado	Flujo completo en dispositivo real.
S6	Cola, historial, errores y espacio	Beta interna utilizable.
S7	Updater firmado y rollback	Actualizacion fallida no rompe la app.
S8	Seguridad, pruebas, accesibilidad y documentacion	Release candidate del MVP.

16.3 Checkpoints obligatorios

Checkpoint A

Antes de implementar autenticacion, demostrar el flujo completo con contenido publico o archivos compartidos.

Checkpoint B

Antes de activar actualizaciones remotas, demostrar firma, hash, staging, prueba y rollback con una version deliberadamente corrupta.

Checkpoint C

Antes de publicar, revisar cada proveedor y eliminar cualquier promesa de acceso universal o reutilizacion de cookies de otra app.

16.4 Riesgos principales

Riesgo	Impacto	Mitigacion
Cambios frecuentes en proveedores	Alto	Adaptadores, actualizador y pruebas de regresion.
Wrapper Android sin mantenimiento	Alto	PoC temprano, fork o runtime propio.
Tamano de la aplicacion	Medio/alto	ABI splits, codecs minimos, medir AAB.
Políticas de Google Play	Alto	Edicion Play separada y revision legal/politica.
Calor y bateria	Medio	Remux, limites y cola secuencial.
Expectativa de contenido privado	Alto	Mensajes claros y matriz de capacidades.
Archivos maliciosos	Alto	Actualizaciones, sandbox, timeouts y validacion.

17. Criterios de aceptacion del MVP

ID	Criterio
AC-001	Bit-Share aparece en el Sharesheet para texto y video en un dispositivo Android limpio.
AC-002	Al compartir una URL valida se abre una interfaz compacta sin iniciar la app completa.
AC-003	La app detecta proveedor o marca el origen como generico sin bloquearse.
AC-004	Una descarga de prueba muestra progreso y puede cancelarse.
AC-005	Al minimizar la app, una descarga larga continua con notificacion visible cuando Android lo permite.
AC-006	El resultado se guarda en la coleccion MediaStore correcta y aparece en galeria/archivos.
AC-007	El resultado puede volver a compartirse sin exponer una ruta privada.
AC-008	FFmpeg une audio y video sin recodificar cuando los codecs son compatibles.
AC-009	Los temporales se eliminan tras exito, cancelacion o fallo controlado.
AC-010	Ningun log contiene cookies, tokens, Authorization ni contraseñas.
AC-011	El motor base funciona sin conexion al servidor de actualizaciones.
AC-012	Una actualizacion con hash incorrecto se rechaza.
AC-013	Una version que falla la prueba activa rollback.
AC-014	La app explica que no puede usar la sesion de otra aplicacion.
AC-015	Las pruebas unitarias y de integracion criticas pasan en CI.

17.1 Definicion de terminado

- Código formateado, analizado y probado.
- Documentacion y CODE_MAP actualizados.
- Sin secretos, binarios no trazables ni dependencias sin licencia revisada.
- Prueba manual en dispositivo arm64 real.
- Capturas y registro de resultados del checkpoint.
- Plan de rollback para cambios de runtime o almacenamiento.

18. Evolucion Windows y Web

18.1 Windows

La version Windows puede reutilizar dominio, adaptadores, historial y motor de tareas, pero tendra una GUI de escritorio. Sus entradas principales pueden ser portapapeles, arrastrar y soltar, extension del navegador, menu contextual y cola masiva. FFmpeg y yt-dlp pueden ejecutarse como procesos controlados con versionado y sandbox de directorios.

- Interfaz de escritorio con tabla de cola y panel de detalles.
- Seleccion avanzada de formatos y perfiles de conversion.
- Integracion con Explorador y portapapeles.
- Sincronizacion futura opcional, nunca necesaria para el nucleo local.

18.2 Web

La Web tiene restricciones distintas: no puede ejecutar binarios nativos tradicionales ni acceder libremente al sistema de archivos. El producto web debe definirse como herramienta complementaria, no como copia exacta de Android.

- Analizar enlaces y generar planes de descarga permitidos.
- Procesar archivos seleccionados con WebAssembly cuando el tamano y navegador lo permitan.
- Usar File System Access API solo como mejora progresiva.
- Evitar prometer descargas autenticadas universales desde el navegador.

18.3 Contratos compartidos

Los paquetes de dominio deben definir SharePayload/InputPayload, ProviderAdapter, MediaFormat, DownloadPlan, TaskState y OutputDestination. Cada plataforma implementara sus propios puentes y motores sin introducir dependencias de Android en el dominio.

19. Prompt maestro para otra IA

Instruccion de uso

Copiar el bloque completo siguiente en la IA de desarrollo. Adjuntar este PDF y el repositorio actual. La IA debe comenzar con analisis y mapa de archivos, no con una reescritura masiva.

Prompt maestro - Parte 1 de 5

ROL Y OBJETIVO

Actua como arquitecto senior de Flutter, Android/Kotlin, seguridad movil, yt-dlp y FFmpeg. Debes construir Bit-Share, una aplicacion de BitStation con un solo repositorio Flutter y experiencias distintas por plataforma. El desarrollo actual es EXCLUSIVAMENTE Android. Windows y Web deben quedar previstos mediante contratos, pero no se implementan todavia.

VISION DEL PRODUCTO

Bit-Share debe aparecer como destino en el menu Compartir de Android. Cuando el usuario comparte una URL, texto o archivo multimedia desde Instagram, YouTube, TikTok, Facebook, X, Twitch, un navegador u otra aplicacion, Bit-Share debe:

1. recibir de forma segura el Intent;
2. identificar el tipo de entrada y proveedor;
3. mostrar una previsualizacion compacta;
4. determinar si el contenido puede descargarse o si la app origen ya compartio el archivo;
5. permitir elegir calidad, formato y destino;
6. descargar o copiar el contenido localmente;
7. procesarlo con FFmpeg cuando sea necesario;
8. preguntar si se guarda en galeria/Descargas, se guarda como archivo o se comparte con otra aplicacion;
9. mantener progreso, cancelacion, reintentos, historial y limpieza de temporales.

RESTRICCION NO NEGOCIABLE SOBRE CREDENCIALES

Android estandar no permite leer cookies, tokens, bases de datos ni cache privada de otras aplicaciones. No implementes root, ADB, Shizuku, Accessibility Service, extraccion de /data/data, lectura de bases de datos de Chrome/Instagram/TikTok, captura de pantalla de contraseñas, hooking, inyeccion ni explotacion de vulnerabilidades.

El Intent de compartir solo entrega datos que la app origen decide compartir. Para contenido autenticado usa un ProviderAdapter independiente con una de estas rutas:

- archivo o URI compartido por la app origen;
- OAuth/SDK/API oficial con scopes concretos;
- sesion propia de Bit-Share solo si el proveedor lo permite y sin capturar la contraseña;
- mensaje NOT_EXPOSED cuando la plataforma no entrega el recurso.

No prometas acceso universal, sin restricciones ni descarga de cualquier contenido privado. No evadas DRM, paywalls ni controles tecnicos. El usuario debe procesar contenido propio o autorizado.

STACK Y ARQUITECTURA

- Flutter/Dart para UI, aplicacion y dominio.
- Kotlin para Android nativo.
- MethodChannel para comandos y EventChannel para eventos.
- WorkManager y/o foreground service para trabajos prolongados visibles.
- MediaStore y Storage Access Framework para resultados.
- Android Keystore para tokens emitidos a Bit-Share.
- yt-dlp mediante una abstraccion DownloaderEngine; no acoples el dominio a un wrapper especifico.
- FFmpeg mediante FfmpegEngine; prioriza remux y stream copy.
- ProviderRegistry con adaptadores por proveedor.
- Persistencia local de cola, historial y estados.

Prompt maestro - Parte 2 de 5

FLUJO DE TRABAJO OBLIGATORIO

1. Inspecciona el repositorio completo antes de modificarlo.
2. Genera y mantiene docs/CODE_MAP.md con carpetas, archivos, responsabilidades y dependencias.
3. No borres ni sobrescribas código existente sin explicar la razón y pedir autorización cuando sea destructivo.
4. Crea cambios pequeños, compilables y verificables.
5. Fija versiones de dependencias y documenta licencias.
6. Tras cada bloque, ejecuta formatter, analyzer, pruebas y build Android.
7. Presenta un checkpoint con: Falla > Mejora > Resumen > Recomendaciones > Feedback esperado.
8. No declares que algo funciona sin adjuntar comandos ejecutados y resultados.

ESTRUCTURA MINIMA

Crea o adapta:

- lib/domain/entities
- lib/domain/repositories
- lib/domain/usecases
- lib/features/share_receiver
- lib/features/media_inspector
- lib/features/download_queue
- lib/features/history
- lib/features/settings
- lib/platform/bitshare_channel.dart
- android/.../share/ShareReceiverActivity.kt
- android/.../bridge/BitSharePlugin.kt
- android/.../download/DownloadCoordinator.kt
- android/.../download/BitShareWorker.kt
- android/.../media/MediaStoreRepository.kt
- android/.../security/SecureTokenVault.kt
- android/.../providers/ProviderRegistry.kt
- docs/ARCHITECTURE.md
- docs/SECURITY.md
- docs/PROVIDER_CONTRACT.md
- docs/UPDATE_PROTOCOL.md
- docs/TEST_PLAN.md
- docs/CODE_MAP.md

MVP POR ORDEN

FASE 0: PoC y decisiones

- Confirmar versión de Flutter, Gradle, Kotlin, NDK y Android SDK.

Prompt maestro - Parte 3 de 5

- Crear PoC real de yt-dlp en arm64-v8a.
- Crear PoC real de FFmpeg.
- Medir tamaño del APK/AAB y tiempo de inicio.
- Registrar ADRs con decisiones y alternativas descartadas.

FASE 1: Receptor de compartir

- Implementar ACTION_SEND y ACTION_SEND_MULTIPLE.
- Aceptar text/plain, image/*, video/* y audio/*.
- Validar Intent, ClipData, MIME, URI, permisos temporales y límites.
- Abrir una ShareReceiverActivity compacta y rápida.

FASE 2: Dominio y proveedores

- Crear SharePayload, AccessLevel, MediaCandidate, MediaFormat, DownloadPlan y DownloadTask.
- Crear ProviderAdapter con inspect(), authorizeIfSupported() y createDownloadPlan().
- Crear ProviderRegistry y GenericProviderAdapter.
- No codificar reglas de proveedor dentro de la UI.

FASE 3: Descarga

- Estrategias en este orden: SharedUriDownloader, DirectHttpDownloader, OfficialApiDownloader, YtDlpDownloader.
- Ejecutar en directorio temporal por taskId.
- Progreso, velocidad, ETA, cancelación, reintento y errores tipados.
- Bloquear localhost, redes privadas, esquemas peligrosos y nombres de archivo inseguros.

FASE 4: FFmpeg y salida

- Merge de video+audio, remux, extracción de audio y transcodificación opcional.
- Priorizar -c copy.
- Publicar resultados mediante MediaStore.
- Compartir con content URI segura; nunca exponer file:///.
- Eliminar temporales y validar que el resultado no esté truncado.

FASE 5: Background y persistencia

- La tarea debe sobrevivir al cierre de Flutter cuando sea razonable.
- Usar notificación visible y acciones Cancelar/Reintentar.
- Persistir estado y rehidratar UI.
- Manejar falta de red, falta de espacio, reinicio y URI caducado.

FASE 6: Actualizador de componentes

- Incluir versión base funcional en la app.

Prompt maestro - Parte 4 de 5

- Actualizar solo componentes permitidos y compatibles.
- Manifiesto HTTPS firmado con Ed25519.
- SHA-256, limites de tamaño, staging, archivos solo lectura, prueba básica, activación atómica y rollback.
- No descargar DEX, JAR, SO, FFmpeg ni runtime Python en la edición Google Play.
- Revisar la política de Google Play antes de activar código interpretado remoto.

FASE 7: Autenticación piloto

- Elegir un solo proveedor con flujo oficial útil para un caso concreto.
- OAuth con PKCE o SDK oficial.
- Guardar token cifrado con Android Keystore.
- Declarar scopes y capacidades reales.
- No convertir OAuth token en cookie ni entregarlo ciegamente a yt-dlp.

SEGURIDAD

- Trata Intents, URLs, metadatos y archivos como no confiables.
- No registres Authorization, cookies, tokens, contraseñas ni URLs firmadas.
- Usa timeouts, límites de tamaño y cancelación cooperativa.
- Verifica integridad de componentes actualizados.
- Genera un threat model y pruebas negativas.
- Incluye opción para borrar historial, sesiones y archivos temporales.

UI ANDROID

Tema oscuro BitStation con acento púrpura y alto contraste. La hoja de compartir debe mostrar:

- miniatura o icono;
- proveedor y título;
- estado de acceso;
- formato/calidad/tamaño;
- Guardar en galería;
- Guardar como archivo;
- Compartir después de descargar;
- Cancelar.

No muestres opciones avanzadas hasta que el usuario las abra.

PRUEBAS Y CRITERIOS

- Unit tests Dart y Kotlin.
- Contract tests para adaptadores.
- Instrumentation tests para Intents y MediaStore.
- E2E: Compartir -> descargar -> FFmpeg -> guardar -> compartir.

Prompt maestro - Parte 5 de 5

- Probar Android 10, 12, 14, 15 y 16; revisar Android 17.
- Probar perdida de red, falta de espacio, cierre, reinicio, actualizacion corrupta y token revocado.

ENTREGABLES DE CADA CHECKPOINT

1. Mapa de archivos actualizado.
2. Lista exacta de archivos nuevos y modificados.
- 3.Codigo completo, sin eliminar bloques anteriores sin permiso.
4. Comandos de compilacion y pruebas ejecutados.
5. Resultado y errores restantes.
6. Riesgos y siguiente paso recomendado.
7. Pregunta de feedback: confirmar si el checkpoint funciona en el dispositivo antes de avanzar.

PRIMERA RESPUESTA ESPERADA

No escribas todo el proyecto de una vez. Primero entrega:

- diagnostico del repositorio;
 - mapa de archivos;
 - decisiones pendientes;
 - PoC propuesto para yt-dlp y FFmpeg;
 - plan de Fase 0 y Fase 1;
 - riesgos de Android/Google Play;
 - comandos concretos para verificar el entorno.
- Luego implementa el primer checkpoint compilable.

20. Plantilla de checkpoint para la IA

Formato reutilizable

```
CHECKPOINT <numero> - <nombre>

FALLA
- Que faltaba, que no compilaba o que riesgo se detecto.

MEJORA
- Cambio implementado y razon tecnica.

RESUMEN
- Archivos nuevos:
- Archivos modificados:
- Interfaces creadas:
- Dependencias agregadas:

VERIFICACION
- flutter doctor -v:
- flutter analyze:
- flutter test:
- ./gradlew test:
- flutter build apk --debug:
- Prueba en dispositivo:

RECOMENDACIONES
- Riesgos restantes.
- Siguiente bloque minimo.

FEEDBACK
- Confirma si Bit-Share aparece en el menu Compartir y si el flujo probado funciona antes de
avanzar.
```

20.1 Registro de decisiones ADR

Plantilla

```
ADR-000X: <titulo>
Estado: Propuesto | Aceptado | Reemplazado
Fecha: YYYY-MM-DD

Contexto
<problema y restricciones>

Decision
<decision adoptada>

Alternativas
<alternativas evaluadas>

Consecuencias
<beneficios, costes y riesgos>

Validacion
<comandos, pruebas o mediciones>
```


21. Referencias tecnicas y normativas

Fuentes oficiales consultadas para esta version. Fecha de consulta: 29 de julio de 2026.

1. Android - Receive simple data from other apps: <https://developer.android.com/training/sharing/receive>
2. Android - Sharing simple data: <https://developer.android.com/training/sharing>
3. Android - Foreground services overview: <https://developer.android.com/develop/background-work/services/fgs>
4. Android - Long-running workers: <https://developer.android.com/develop/background-work/background-tasks/persistent/how-to/long-running>
5. Android - Data transfer background task options: <https://developer.android.com/develop/background-work/background-tasks/data-transfer-options>
6. Android - Access media files from shared storage: <https://developer.android.com/training/data-storage/shared/media>
7. Android - Storage Access Framework: <https://developer.android.com/training/data-storage/shared/documents-files>
8. Android - Privacy and app sandbox: <https://developer.android.com/privacy>
9. Android - Dynamic code loading security: <https://developer.android.com/privacy-and-security/risks/dynamic-code-loading>
10. Android - Security checklist: <https://developer.android.com/privacy-and-security/security-tips>
11. Flutter - Platform channels: <https://docs.flutter.dev/platform-integration/platform-channels>
12. Flutter - Developing packages and plugins: <https://docs.flutter.dev/packages-and-plugins/developing-packages>
13. yt-dlp - Official repository: <https://github.com/yt-dlp/yt-dlp>
14. yt-dlp - Installation wiki: <https://github.com/yt-dlp/yt-dlp/wiki/Installation>
15. Google Play - Developer Program Policy: <https://support.google.com/googleplay/android-developer/answer/17190352>

21.1 Nota sobre fuentes de plataformas sociales

Las capacidades de OAuth y APIs de YouTube/Google, Meta, TikTok, Twitch y X cambian con frecuencia y dependen de permisos, revision de aplicaciones, region, tipo de cuenta y terminos. Antes de implementar cada ProviderAdapter se debe consultar exclusivamente su documentacion oficial vigente y registrar la fecha, scopes, endpoints, limites y uso permitido en docs/PROVIDER_CONTRACT.md.

21.2 Conclusión final

Direccion recomendada

Construir primero una herramienta Android excelente para recibir, descargar contenido compatible, convertir, guardar y compartir. Mantener la autentificacion como capacidad modular y honesta. El exito de Bit-Share depende mas de una arquitectura adaptable y una experiencia clara que de intentar reutilizar sesiones privadas que Android deliberadamente aisla.